SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code	**CSA4315**	**Course Title**	**Internet Programming**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Data Interpretation Based Questions**
Weightage	**30% of CIA**	**Total Marks**	**100**
CO1 Statement	*Explain the fundamentals of web architecture, client-server communication, Internet protocols, HTTP request and response messages, and configure LAMP/WAMP environments for web application development and deployment.* **(BL3)**		
Student Name	**Papudesi Mohitha**	**Reg. No.**	**192324025**
Section/Batch	**Slot D / 2023**	**Date**	**20-10-2026**

Code of Conduct:

I, Papudesi Mohitha, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P. Mohitha

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Calculation Accuracy (Parts a & b)	**30%**	Both totals (CPU = 75%, memory = 2,400 MB) are correct, with complete step-by-step working, units and the additive assumption stated.	Both totals are correct with working shown, but units or the stated assumption is missing.	One total is correct, or both have minor arithmetic or unit errors.	Totals are incorrect or no working is shown.	
Data Interpretation (Part c)	**20%**	Correctly identifies MySQL (600 MB) as the highest memory consumer, compares all three components and gives percentage shares.	Correctly identifies MySQL with a basic comparison of the three values.	Identifies MySQL but gives no comparison or reasoning.	Identifies the wrong component or gives no answer.	
Performance Recommendations (Part d)	**25%**	Two relevant, practical and justified improvements, each tied to the observed data and with implementation steps.	Two relevant improvements with justification but limited link to the data.	Only one valid improvement, or two generic ones without justification.	Irrelevant or missing suggestions.	
Technical Understanding of LAMP/WAMP	**15%**	Clearly explains the roles of Apache, MySQL and PHP and how resource use relates to the web request cycle.	Explains the roles of the components with minor gaps.	Gives a partial or vague explanation of the components.	Shows no understanding of the LAMP/WAMP stack.	
Presentation & Organisation	**10%**	Well-structured, uses tables and headings, is free of errors and meets the length requirement.	Well organised with minor formatting or language issues.	Partially organised, with several formatting or language issues.	Poorly organised, hard to follow.	

SECTION B – DATA INTERPRETATION-BASED QUESTIONS

Question 25: LAMP/WAMP Resource Utilization

1. Question Statement

A developer observes the following resource utilization on a local development computer running a LAMP/WAMP web development stack.

Component	CPU utilization	Memory usage
Apache	18%	350 MB
MySQL	25%	600 MB
PHP processing	12%	250 MB
Other applications	20%	1,200 MB

Table 1: Observed resource utilization

Tasks: (a) calculate the combined CPU utilization; (b) calculate the total memory usage; (c) identify the component consuming the most memory among Apache, MySQL and PHP processing; (d) suggest two ways to improve the computer's performance.

2. Introduction and Background

LAMP stands for Linux, Apache, MySQL and PHP, while WAMP is the same stack running on Windows. Together these components allow a developer to build, test and deploy dynamic web applications on a local computer before they are moved to a production server. Each component has a clearly defined role in the web request cycle.

- **Apache (web server):** receives HTTP requests from the browser (the client), decides which resource is needed and returns an HTTP response. It creates worker processes or threads to handle simultaneous connections, so its CPU and memory use grows with the number of requests.
- **PHP (server-side scripting):** executes the application code when a dynamic page is requested. It interprets the script, talks to the database and generates the HTML that Apache sends back. PHP consumes CPU while scripts run and memory for each script's variables and loaded libraries.
- **MySQL (database server):** stores and retrieves application data. It keeps buffers and caches (such as the InnoDB buffer pool) in RAM to avoid slow disk reads, which is why a database server is normally the largest memory consumer in the stack.
- **Other applications:** browsers, code editors, messaging tools and background services that run alongside the stack and compete for the same CPU and RAM.

Because all of these programs share a single machine, the combined resource usage determines how smooth the development experience is. Interpreting the measured data correctly helps the developer to identify bottlenecks and tune the system. Resource utilization data such as that in Table 1 is a basic example of data interpretation in web application deployment (BL3 – Apply).

3. Part (a): Combined CPU Utilization

Assumption: as stated in the question, all measurements are taken over the same time interval and are additive. Therefore, the total CPU utilization is the sum of the individual percentages.

Given: Apache = 18%, MySQL = 25%, PHP processing = 12%, Other applications = 20%.

Total CPU utilization = Apache + MySQL + PHP + Other applications

= 18% + 25% + 12% + 20%

= 43% + 12% + 20%

= 55% + 20% = 75%

Answer: the combined CPU utilization of all listed processes is **75%**.

Interpretation

- The stack itself (Apache + MySQL + PHP) uses 18 + 25 + 12 = 55% of the CPU, and the other applications add 20%.
- The remaining unused (idle) capacity is 100% − 75% = 25%. A headroom of only one quarter of the CPU means that any sudden load, such as several simultaneous page requests or a complex query, can push the system towards saturation and cause visible slowdowns.
- MySQL is the largest CPU consumer at 25%, followed by Other applications at 20%, Apache at 18% and PHP at 12%.

The share of each component in the total 75% CPU utilization is given below.

Component	CPU (%)	Share of used CPU	Share of total CPU
Apache	18	24.00%	18%
MySQL	25	33.33%	25%
PHP processing	12	16.00%	12%
Other applications	20	26.67%	20%
Total (used)	**75**	**100%**	**75%**

Table 2: CPU share analysis (Share of used CPU = component ÷ 75 × 100)

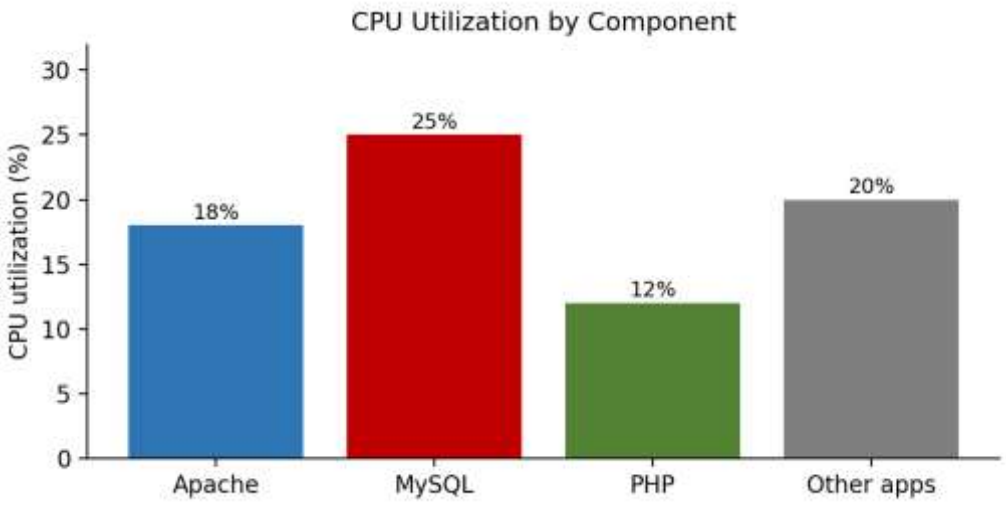


Figure 1: CPU utilization by component

4. Part (b): Total Memory Usage

Given: Apache = 350 MB, MySQL = 600 MB, PHP processing = 250 MB, Other applications = 1,200 MB.

$$\textbf{Total memory usage} = 350 + 600 + 250 + 1{,}200$$
$$= 950 + 250 + 1{,}200$$
$$= 1{,}200 + 1{,}200 = 2{,}400 \text{ MB}$$

Answer: the total memory usage is **2,400 MB**, which is approximately 2.34 GB (2,400 ÷ 1,024) or 2.4 GB if 1 GB is taken as 1,000 MB.

Interpretation

- The LAMP/WAMP stack alone uses 350 + 600 + 250 = 1,200 MB, which is exactly half of the total memory in use.
- The other applications also use 1,200 MB, i.e. 50% of the total. These background programs therefore consume as much memory as the entire web stack, which is a major opportunity for optimisation.
- A typical development laptop has 4 GB, 8 GB or 16 GB of RAM. With 2.4 GB already in use, a machine with 4 GB would be left with little free memory and may start swapping to disk, which makes everything slower. An 8 GB machine would remain comfortable.

Component	Memory (MB)	Share of total memory	Memory (GB, ÷1024)
Apache	350	14.58%	0.34
MySQL	600	25.00%	0.59
PHP processing	250	10.42%	0.24
Other applications	1,200	50.00%	1.17
Total	**2,400**	**100%**	**2.34**

Table 3: Memory share analysis (Share = component ÷ 2,400 × 100)

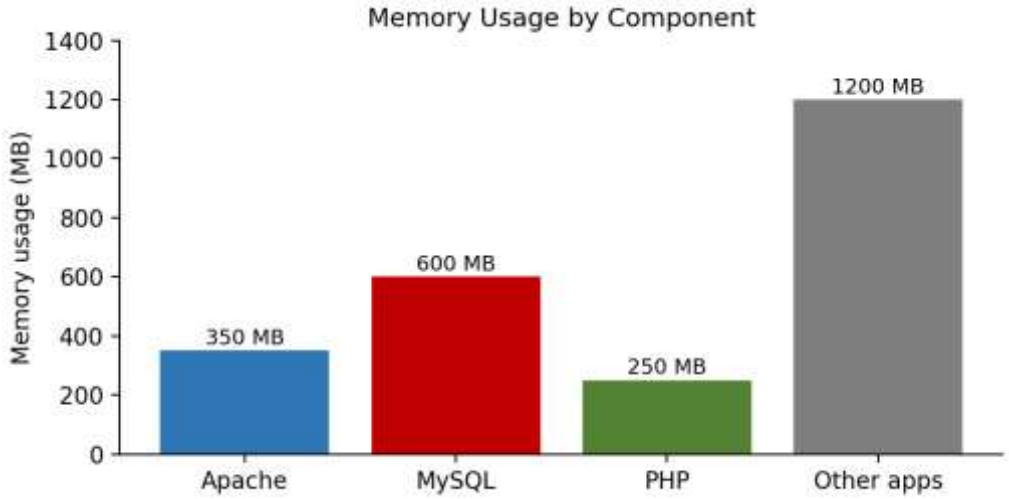


Figure 2: Memory usage by component

5. Part (c): Component Consuming the Most Memory

Only Apache, MySQL and PHP processing are to be compared. The values are:

Component	Memory (MB)	Rank
MySQL	600	1 (highest)
Apache	350	2
PHP processing	250	3 (lowest)

Table 4: Memory ranking of the LAMP/WAMP components

Answer: MySQL consumes the most memory among the three, at 600 MB.

Comparison

- The three components together use 1,200 MB. MySQL's 600 MB is therefore 600 ÷ 1,200 = 50% of the stack's memory, Apache's 350 MB is 29.17% and PHP's 250 MB is 20.83%.
- MySQL uses 250 MB (71.4%) more than Apache and 350 MB (140%) more than PHP.

Why MySQL uses the most memory

- MySQL keeps frequently used table data and indexes in memory (the InnoDB buffer pool) so queries do not need to read from the disk each time.
- It allocates memory for connection threads, sort buffers, join buffers and the query cache, all of which grow with the number of connections and the size of the queries.
- MySQL is also a long-running service which keeps its caches loaded even when the application is idle, whereas PHP releases the memory used by a script when the request finishes.

This high memory use is normal and even desirable to a certain extent, because it makes database access faster. The aim of tuning is therefore not to remove the buffers but to size them sensibly for a development environment, where the data set and the number of users are small.

6. Part (d): Two Ways to Improve Performance

The data shows that CPU headroom is only 25% and that half of the memory is consumed by non-stack applications. The following two improvements address these observations directly.

Improvement 1: Close or limit unnecessary background applications

The “Other applications” category uses 20% of the CPU and 1,200 MB of memory, which is the largest single block of memory in the system and is not required for web development. The developer should:

- open the Task Manager (Windows) or a monitor such as top/htop (Linux) and identify programs that are not needed during development;
- close extra browser tabs and extensions, heavy chat or video applications and unused IDE windows;
- disable unnecessary start-up programs and background services so they do not load every time the computer boots.

Expected benefit: if the other applications can be reduced by even half, about 600 MB of RAM and 10% of the CPU are released. This would bring total usage down to roughly 1,800 MB and 65% CPU, and the stack would respond faster because it is no longer competing for resources.

Improvement 2: Tune the LAMP/WAMP configuration and upgrade hardware if required

The stack components themselves use 55% of the CPU and 1,200 MB of RAM. They can be made lighter by configuration changes that suit a single-developer environment:

- **MySQL:** reduce innodb_buffer_pool_size to a value that fits the local data set, lower max_connections, and add indexes to frequently queried columns so that queries use less CPU.
- **Apache:** reduce the number of worker processes (StartServers, MaxRequestWorkers) and switch off modules that are not used, so fewer idle workers occupy memory.
- **PHP:** enable OPcache so that compiled scripts are cached and not re-parsed on every request, and lower memory_limit to a realistic value.
- **Hardware:** if the machine still runs slowly after tuning, adding RAM (for example from 4 GB to 8 GB) and moving from a hard disk to an SSD gives the largest improvement for database-heavy development work.

Expected benefit: lower memory and CPU use by the stack, faster page loads and quicker database queries, and more free capacity for handling peaks in load.

Before and after comparison (estimated)

The table below shows an illustrative estimate of the effect of the two improvements. The values are assumptions for explanation only, not measurements.

Resource	Current	After improvements (estimate)	Change
Total CPU utilization	75%	≈ 55%	− 20%
Total memory usage	2,400 MB	≈ 1,700 MB	− 700 MB
Idle CPU headroom	25%	≈ 45%	+ 20%

Table 5: Estimated effect of the proposed improvements

7. Summary of Answers

Part	Question	Answer
(a)	Combined CPU utilization	75%
(b)	Total memory usage	2,400 MB (≈ 2.34 GB)
(c)	Highest memory consumer among Apache, MySQL and PHP	MySQL (600 MB)
(d)	Two ways to improve performance	1. Close unnecessary background applications 2. Tune the stack configuration / upgrade hardware

Table 6: Final answers

8. Conclusion

The analysis shows that the local development computer is using 75% of its CPU and 2,400 MB of memory. Although the LAMP/WAMP components are all necessary, they account for only 55% of the CPU and half of the memory, and the remaining half is consumed by applications unrelated to web development. Among the stack components, MySQL is the biggest memory consumer at 600 MB because of the buffers and caches it keeps in RAM. Closing unneeded background applications and tuning Apache, MySQL and PHP (with a hardware upgrade where necessary) will free CPU and memory, leave more headroom for peak loads and give the developer a faster and more reliable environment for building and testing web applications.